B.TECH CSE — $5^{th}$ SEMESTER LECTURE NOTES

# Chapter 4

## Knowledge Representation and Logical Reasoning

---

Based on: *Artificial Intelligence and Machine Learning*

Pradeep Singh, Tapan K. Gandhi, Balasubramanian Raman




---

# Contents


**4 Knowledge Representation and Logical Reasoning** **1**

4.1 Introduction to Knowledge Representation . . . . . 1

4.1.1 Why do we need Knowledge Representation? . . . . . 1

4.1.2 Types of Knowledge . . . . . 2

4.1.3 Properties a Good KR System Should Have . . . . . 2

4.2 Approaches / Techniques of Knowledge Representation . . . . . 2

4.2.1 Semantic Networks . . . . . 3

4.2.2 Frame Representation . . . . . 3

4.2.3 Production Rules (Rule-Based Systems) . . . . . 4

4.3 Propositional Logic (PL) . . . . . 4

4.3.1 Syntax: Connectives . . . . . 4

4.3.2 Truth Table (build once, reuse always) . . . . . 5

4.3.3 Well-Formed Formula (WFF), Tautology, Contradiction . . . . . 5

4.3.4 Logical Equivalences (Laws) . . . . . 6

4.3.5 Rules of Inference . . . . . 6

4.3.6 Converting to Conjunctive Normal Form (CNF) . . . . . 7

4.3.7 Resolution Refutation (Proof by Contradiction) . . . . . 8

4.4 First-Order (Predicate) Logic (FOL) . . . . . 9

4.4.1 Quantifiers . . . . . 9

4.4.2 Translating English to FOL . . . . . 9

4.4.3 Unification . . . . . 10

4.4.4 Resolution in FOL . . . . . 11

4.5 Forward Chaining and Backward Chaining . . . . . 11

4.6 Comparison: Propositional Logic vs. First-Order Logic . . . . . 13

4.7 Ontology . . . . . 13

4.8 RDF and OWL: Basic Ideas . . . . . 14

4.9 Knowledge Representation Using Scripts . . . . . 15

4.10 Common Pitfalls . . . . . 15

4.11 Practice Problems for Self-Study . . . . . 15

4.12 Cheat Sheet . . . . . 17

# Knowledge Representation and Logical Reasoning

**What this chapter covers:** How an intelligent agent stores facts about the world (knowledge representation) and how it derives new facts from old ones (logical reasoning) — propositional logic, first-order (predicate) logic, inference rules, resolution, forward/backward chaining, and structured representations (semantic nets, frames, scripts).

**Why it matters:** Every "expert system", rule engine, chatbot with reasoning, or planning module ultimately reduces to *representing* facts and *inferring* new ones from them. This chapter is the mathematical backbone of classical (symbolic) AI, and is a favourite for university exams because it mixes theory with short hand-solvable problems.

## 4.1 Introduction to Knowledge Representation

An intelligent agent needs to **know** things about the world – facts, rules, relationships – and needs a way to **store** and **manipulate** that knowledge so that it can reason and make decisions.

**Definition 1:** Knowledge Representation (KR) is the part of AI concerned with how knowledge about the world can be represented inside a machine, in a form that a reasoning/inference program can use to derive new conclusions.

A formal knowledge representation system may be viewed as:

$$\mathcal{K} = (S, \mathcal{L}, \mathcal{I}),$$

where $S$ is a set of symbols, $\mathcal{L}$ is a language for forming statements, and $\mathcal{I}$ gives meaning to those statements.

### 4.1.1 Why do we need Knowledge Representation?

- Raw data (pixels, numbers, text) is not directly usable for reasoning – it must be converted into a symbolic/structured form.
- A good KR scheme lets an agent **infer** new facts instead of storing every possible

fact explicitly.

- It allows knowledge to be **shared, reused, updated** and **explained** (important for expert systems).

### 4.1.2 Types of Knowledge

| Type | Description & Example |
|---|---|
| **Declarative** | "Knowing that" – static facts. *E.g. "Delhi is the capital of India."* |
| **Procedural** | "Knowing how" – steps to perform a task. *E.g. how to solve a quadratic equation.* |
| **Meta-knowledge** | Knowledge about knowledge – what we know and how reliable it is. *E.g. "Rule R1 is 90% reliable."* |
| **Heuristic** | Rules of thumb / experience-based knowledge used to guide search. *E.g. "Try the move that captures the opponent's queen first."* |
| **Structural** | Knowledge about how concepts relate to one another (categories, part-of relations). *E.g. "A car has an engine, wheels..."* |

### 4.1.3 Properties a Good KR System Should Have

1. **Representational Adequacy** – ability to represent all kinds of knowledge needed in the domain.
2. **Inferential Adequacy** – ability to derive new knowledge from existing knowledge.
3. **Inferential Efficiency** – ability to guide the inference process efficiently (using heuristics).
4. **Acquisitional Efficiency** – ability to easily acquire new knowledge (learning).

> **Exam tip:** These four properties (*Representational Adequacy, Inferential Adequacy, Inferential Efficiency, Acquisitional Efficiency*) are a very common 2-mark / short-answer question. Remember them with the acronym **RIIA**.

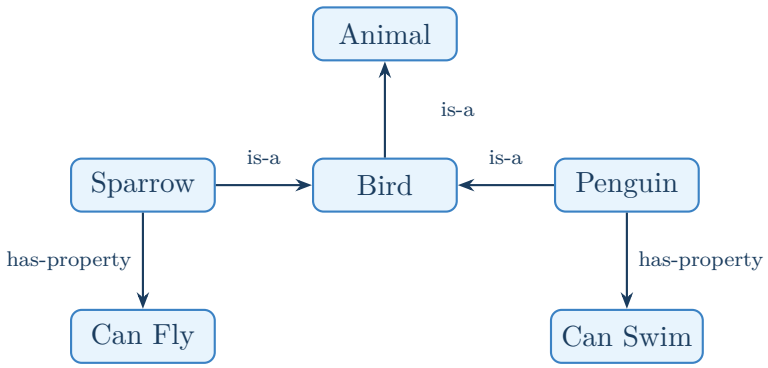
## 4.2 Approaches / Techniques of Knowledge Representation

There are four broad approaches used to represent knowledge in AI systems.

| Approach | Idea |
|---|---|
| **Logical Representation** | Facts expressed as logical statements (propositional / predicate logic) with precise syntax and semantics; supports formal proof. |
| **Semantic Network** | Knowledge shown as a labelled graph – nodes are objects/concepts, edges are relationships (*is-a*, *has-a*, etc.). |
| **Frame Representation** | Knowledge organised as a record-like structure (*frame*) with named *slots* holding attribute values, similar to an object in OOP. |
| **Production Rules** | Knowledge expressed as **IF (condition) THEN (action)** rules, used heavily in expert systems. |

### 4.2.1 Semantic Networks

A **semantic network** represents knowledge as a directed graph. Nodes represent objects, concepts, or events; arcs (edges) represent the relationship between them.




**Property inheritance:** In a semantic network, a subclass automatically inherits properties of its superclass unless explicitly overridden. Above, *Sparrow* inherits "is an Animal" from *Bird*, but has its own extra property *Can Fly* (Penguin does not inherit this – exceptions are stored locally).

### 4.2.2 Frame Representation

A **frame** is a data structure that divides knowledge into substructures called **slots**, each holding a value, a default, or a pointer to a procedure.

| Frame: Student | |
|---|---|
| Slot | Value |
| Name | Ramesh |
| Roll_No | CSE21045 |
| Department | Computer Science |
| CGPA | 8.4 |
| Enrolled_Courses | {AI, DBMS, OS} |

### 4.2.3 Production Rules (Rule-Based Systems)

Knowledge is written as **IF–THEN** rules, forming the core of **expert systems**. A production system has three parts:

1. **Global database / working memory** – current facts.
2. **Set of production rules** – `IF <condition> THEN <action>`.
3. **Control / rule interpreter** – decides which rule fires (*match–select–act cycle*).

*Example rule:* `IF (patient has fever) AND (patient has rash) THEN (suspect measles)`.

## 4.3 Propositional Logic (PL)

**Definition 2:** Propositional Logic is the simplest form of logic in which statements (called **propositions**) are either **True (T)** or **False (F)**, and complex statements are built from simple ones using logical connectives.

### 4.3.1 Syntax: Connectives

| Connective | Symbol | Meaning |
|---|---|---|
| Negation | $\neg P$ | NOT $P$ |
| Conjunction | $P \wedge Q$ | $P$ AND $Q$ |
| Disjunction | $P \vee Q$ | $P$ OR $Q$ |
| Implication | $P \rightarrow Q$ | IF $P$ THEN $Q$ |
| Biconditional | $P \leftrightarrow Q$ | $P$ IF AND ONLY IF $Q$ |

### 4.3.2 Truth Table (build once, reuse always)

| $P$ | $Q$ | $\neg P$ | $P \wedge Q$ | $P \vee Q$ | $P \to Q$ | $P \leftrightarrow Q$ |
|---|---|---|---|---|---|---|
| T | T | F | T | T | T | T |
| T | F | F | F | T | F | F |
| F | T | T | F | T | T | F |
| F | F | T | F | F | T | T |

**Remember:** $P \to Q$ is **False only when** $P$ is True and $Q$ is False. In every other case it is True. Students often wrongly mark $F \to F$ as false – it is actually **True** ("vacuously true").

### 4.3.3 Well-Formed Formula (WFF), Tautology, Contradiction

- A **tautology** is a formula that is True for *every* combination of truth values (e.g. $P \vee \neg P$).
- A **contradiction** is a formula that is False for every combination (e.g. $P \wedge \neg P$).
- A **contingency** is neither – true for some rows, false for others.

**Solved Problem 1:** Truth table and tautology check. Show that $(P \to Q) \leftrightarrow (\neg P \vee Q)$ is a tautology.

*Solution:*

| $P$ | $Q$ | $\neg P$ | $P \to Q$ | $\neg P \vee Q$ | $(P \to Q) \leftrightarrow (\neg P \vee Q)$ |
|---|---|---|---|---|---|
| T | T | F | T | T | **T** |
| T | F | F | F | F | **T** |
| F | T | T | T | T | **T** |
| F | F | T | T | T | **T** |

Since the last column is **True in every row**, the formula is a **tautology**. This identity, $P \to Q \equiv \neg P \vee Q$, is extremely useful and is used later to convert formulas to CNF.

### 4.3.4 Logical Equivalences (Laws)

| Law | Equivalence |
|---|---|
| Double negation | $\neg(\neg P) \equiv P$ |
| De Morgan's Laws | $\neg(P \wedge Q) \equiv \neg P \vee \neg Q \quad ; \quad \neg(P \vee Q) \equiv \neg P \wedge \neg Q$ |
| Distributive | $P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$ |
| Implication elimination | $P \to Q \equiv \neg P \vee Q$ |
| Biconditional elimination | $P \leftrightarrow Q \equiv (P \to Q) \wedge (Q \to P)$ |
| Contrapositive | $P \to Q \equiv \neg Q \to \neg P$ |

### 4.3.5 Rules of Inference

**Modus Ponens:** From $P \to Q$ and $P$, infer $Q$.

**Modus Tollens:** From $P \to Q$ and $\neg Q$, infer $\neg P$.

**Hypothetical Syllogism:** From $P \to Q$ and $Q \to R$, infer $P \to R$.

**Disjunctive Syllogism:** From $P \vee Q$ and $\neg P$, infer $Q$.

**Addition:** From $P$, infer $P \vee Q$.

**Simplification:** From $P \wedge Q$, infer $P$.

**Conjunction:** From $P$ and $Q$, infer $P \wedge Q$.

**Resolution:** From $P \vee Q$ and $\neg P \vee R$, infer $Q \vee R$.

**Solved Problem 2:** Let:

$$p = \text{It is raining}, \qquad q = \text{Road is wet}.$$

Given:

$$p \to q$$

and

$$p,$$

Modus Ponens gives:

$$\boxed{q}.$$

Therefore, the road is wet.

**Solved Problem 3:** Applying rules of inference (a classic exam question). Given the premises:

(1) If it rains, the match is cancelled: $R \to C$

(2) If the match is cancelled, the team is disappointed: $C \to D$

(3) It rains: $R$

Prove that "the team is disappointed" ($D$).

*Solution:*

1. From (1) and (2), by **Hypothetical Syllogism**: $R \to D$.
2. From step 1 and (3), by **Modus Ponens**: $D$.

Hence $D$ **(the team is disappointed) is proved.** ■

### 4.3.6 Converting to Conjunctive Normal Form (CNF)

Resolution needs sentences in **CNF** – a conjunction of clauses, where each clause is a disjunction of literals.

**Steps to convert a formula to CNF:**

1. Eliminate $\leftrightarrow$: replace $P \leftrightarrow Q$ with $(P \to Q) \wedge (Q \to P)$.
2. Eliminate $\to$: replace $P \to Q$ with $\neg P \vee Q$.
3. Push $\neg$ inward using De Morgan's laws until it applies only to atoms.
4. Distribute $\vee$ over $\wedge$ to get a conjunction of disjunctions.

**Solved Problem 4:** Convert to CNF. Convert $\neg(P \wedge Q) \to R$ to CNF.

*Solution:*

$$\begin{aligned} \neg(P \wedge Q) \to R &\equiv \neg[\neg(P \wedge Q)] \vee R && \text{(eliminate } \to\text{)} \\ &\equiv (P \wedge Q) \vee R && \text{(double negation)} \\ &\equiv (P \vee R) \wedge (Q \vee R) && \text{(distribute } \vee \text{ over } \wedge\text{)} \end{aligned}$$

**CNF form:** $(P \vee R) \wedge (Q \vee R)$, i.e. two clauses: $\{P, R\}$ and $\{Q, R\}$.

### 4.3.7 Resolution Refutation (Proof by Contradiction)

**Algorithm:**

1. Convert all premises *and the negation of the conclusion* to CNF (a set of clauses).
2. Repeatedly resolve pairs of clauses that contain complementary literals ($L$ and $\neg L$), producing a new clause (the resolvent).
3. If the **empty clause** ($\square$ / contradiction) is derived, the original conclusion is **proved**.

**Solved Problem 5:** Resolution refutation proof. Given: (1) $P \to Q$ (2) $Q \to R$ (3) $P$. Prove $R$.

*Solution:* **Step 1 – Convert premises and $\neg R$ to clause form:**

$$\begin{aligned} &(1)\ P \to Q && \equiv \neg P \vee Q \\ &(2)\ Q \to R && \equiv \neg Q \vee R \\ &(3)\ P && \equiv P \\ &\text{Negated goal: } \neg R && \equiv \neg R \end{aligned}$$

**Step 2 – Resolve:**

1. Resolve $(\neg P \vee Q)$ with $P \Rightarrow Q$
2. Resolve $Q$ with $(\neg Q \vee R) \Rightarrow R$
3. Resolve $R$ with $\neg R \Rightarrow$ **empty clause** $\square$

Since we reached the empty clause, the negated goal leads to a contradiction $\Rightarrow R$ **is proved.** ■

## 4.4 First-Order (Predicate) Logic (FOL)

Propositional logic cannot express relationships between objects or generalisations like "*all* humans are mortal". **First-Order Logic** extends it with **objects, predicates, functions, and quantifiers**.

**Definition 3:** First-Order Logic represents knowledge in terms of **objects** (constants: `Ram`, `5`), **predicates** (relations: `Likes(Ram, Mango)`), **functions** (`FatherOf(Ram)`), **connectives** (same as PL), and **quantifiers** ($\forall$, $\exists$).

### 4.4.1 Quantifiers

| Quantifier | Meaning |
|---|---|
| **Universal** $\forall x$ | "for all $x$" – true for every object in the domain. *E.g.* $\forall x\ Human(x) \rightarrow Mortal(x)$: "All humans are mortal." |
| **Existential** $\exists x$ | "there exists an $x$" – true for at least one object. *E.g.* $\exists x\ Student(x) \wedge Likes(x, AI)$: "Some student likes AI." |

**Golden rule:** $\forall$ is normally paired with $\rightarrow$ (implication), and $\exists$ is normally paired with $\wedge$ (conjunction). Using $\forall x\, P(x) \wedge Q(x)$ instead of $\forall x\, P(x) \rightarrow Q(x)$ is one of the most common student mistakes – it wrongly claims *everything* in the universe satisfies $P$.

### 4.4.2 Translating English to FOL

**Solved Problem 6:** Translate English sentences into FOL.

(a) "All dogs are animals."
$\forall x\ Dog(x) \rightarrow Animal(x)$

(b) "Some students are intelligent."
$\exists x\ Student(x) \wedge Intelligent(x)$

(c) "Every student who studies passes the exam."
$\forall x\ (Student(x) \wedge Studies(x)) \rightarrow Passes(x)$

(d) "Ram likes every fruit that is sweet."
$\forall x\ (Fruit(x) \wedge Sweet(x)) \rightarrow Likes(Ram, x)$

(e) "Nobody loves everybody."
$\neg\exists x\, \forall y\, L(x, y) \equiv \forall x\, \exists y\, \neg L(x, y)$

### 4.4.3 Unification

Unification finds substitutions that make two logical expressions identical.

Suppose:

$$P(a, y)$$

and

$$P(x, b).$$

A unifier is:

$$\boxed{\theta = \{x \mapsto a,\ y \mapsto b\}}.$$

After substitution, both become:

$$P(a, b).$$

**Solved Problem 7:** Unify:

$$Parent(x, John)$$

and

$$Parent(Alice, y).$$

Choose:

$$x = Alice, \qquad y = John.$$

Thus the most general unifier is:

$$\boxed{\theta = \{x/Alice,\ y/John\}}.$$

**Solved Problem 8:** Find the Most General Unifier (MGU). Unify $Likes(x, Mango)$ and $Likes(Ram, y)$.

*Solution:* Match position by position:

- Position 1: $x$ (variable) vs. $Ram$ (constant) $\Rightarrow$ substitute $x/Ram$
- Position 2: $Mango$ (constant) vs. $y$ (variable) $\Rightarrow$ substitute $y/Mango$

**MGU** $= \{x/Ram,\ y/Mango\}$. Applying it to both predicates gives the identical term $Likes(Ram, Mango)$.

**Solved Problem 9:** Unification that fails.
Unify $Likes(x, x)$ and $Likes(Ram, Shyam)$.

*Solution:* Position 1 forces $x = Ram$; position 2 forces $x = Shyam$. Since $x$ cannot simultaneously equal two different constants, **unification fails**.

### 4.4.4 Resolution in FOL

The same resolution principle from propositional logic applies, but literals are unified (using MGU) before cancelling complementary pairs.

**Solved Problem 10:** FOL Resolution. Premises:

(1) $\forall x\ Man(x) \rightarrow Mortal(x)$

(2) $Man(Socrates)$

Prove: $Mortal(Socrates)$.

*Solution:* Clause form: (1) $\neg Man(x) \vee Mortal(x)$; (2) $Man(Socrates)$; negated goal (3) $\neg Mortal(Socrates)$.

1. Unify $Man(x)$ in (1) with $Man(Socrates)$ in (2) using $\{x/Socrates\}$, resolve $\Rightarrow Mortal(Socrates)$.
2. Resolve $Mortal(Socrates)$ with $\neg Mortal(Socrates)$ (3) $\Rightarrow$ **empty clause**.

Contradiction reached $\Rightarrow$ $Mortal(Socrates)$ **is proved.** This is exactly the classical "Socrates is mortal" syllogism, now proved mechanically. ■

## 4.5 Forward Chaining and Backward Chaining

These are two strategies used by rule-based (production) systems to derive conclusions from IF–THEN rules.

| Aspect | Forward Chaining | Backward Chaining |
|---|---|---|
| **Direction** | Data → Goal (data-driven) | Goal → Data (goal-driven) |
| **Starts from** | Known facts | The query / hypothesis to be proved |
| **Used in** | Expert systems that monitor and react (e.g. diagnosis, planning) | Systems that answer a specific query (e.g. Prolog, expert-system consultation) |
| **Nature** | Bottom-up | Top-down |

**Solved Problem 11:** Forward chaining. **Rules:**

R1: $A \wedge B \to C$

R2: $C \to D$

R3: $D \wedge E \to F$

**Known facts:** $A$, $B$, $E$. **Goal:** prove $F$.

*Solution (data-driven, apply any rule whose left side is satisfied):*

1. Facts $\{A, B, E\}$: R1 fires ($A \wedge B$ known) $\Rightarrow$ add $C$. Facts $= \{A, B, C, E\}$.
2. R2 fires ($C$ known) $\Rightarrow$ add $D$. Facts $= \{A, B, C, D, E\}$.
3. R3 fires ($D \wedge E$ known) $\Rightarrow$ add $F$. Facts $= \{A, B, C, D, E, F\}$.

$F$ has been derived $\Rightarrow$ **goal reached.**

**Solved Problem 12:** Backward chaining (same rule base as above). **Goal:** prove $F$.

*Solution (goal-driven, work backwards to find what supports the goal):*

1. To prove $F$, need R3: requires $D \wedge E$. $E$ is a known fact $\Rightarrow$ need to prove $D$.
2. To prove $D$, need R2: requires $C \Rightarrow$ need to prove $C$.
3. To prove $C$, need R1: requires $A \wedge B$. Both $A$ and $B$ are known facts $\Rightarrow$ $C$ is proved.
4. Back-substitute: $C$ true $\Rightarrow$ $D$ true (R2) $\Rightarrow$ $F$ true (R3, since $E$ known).

$F$ **is proved** – notice backward chaining only explored rules relevant to $F$, which is more efficient when there are many irrelevant rules.

## 4.6 Comparison: Propositional Logic vs. First-Order Logic

| Feature | Propositional Logic | First-Order Logic |
|---|---|---|
| **Basic unit** | Whole statement (proposition) | Objects, predicates, functions |
| **Quantifiers** | Not available | $\forall$, $\exists$ available |
| **Expressiveness** | Limited | Much higher; can express relations |
| **Example** | $P$: "It is raining" | $Rain(today)$ |

## 4.7 Ontology

An **ontology** formally describes the concepts in a domain, their relationships, and the rules that connect them. It provides a shared vocabulary for representing knowledge.

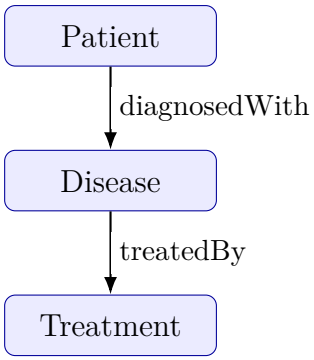
A simplified ontology can be represented as:

$$\mathcal{O} = (C, R, I, A),$$

where:

- $C$: **concepts/classes**, such as Patient, Disease and Treatment;
- $R$: **relationships**, such as `diagnosedWith` and `treatedBy`;
- $I$: **instances**, such as Alice, a particular patient;
- $A$: **axioms**, such as "Every Doctor is a Person."

For example, a small healthcare ontology can connect the following classes:




The diagram shows the kinds of relationships that may hold between instances of these classes. For example, Alice may be diagnosed with influenza, which may be treated with a suitable treatment.

## 4.8 RDF and OWL: Basic Ideas

### RDF: Representing Facts

**RDF (Resource Description Framework)** represents information using triples:

$$\boxed{(\text{Subject},\ \text{Predicate},\ \text{Object})}$$

For example:

$$(\text{Alice},\ \text{diagnosedWith},\ \text{Influenza}).$$

Here:

- **Subject**: Alice;
- **Predicate**: `diagnosedWith`;
- **Object**: Influenza.

The triple states that Alice is diagnosed with influenza. In actual RDF, resources and properties are identified using IRIs; the notation above is a simplified illustration.

### OWL: Describing Classes and Inferring Knowledge

**OWL (Web Ontology Language)** builds on RDF to express richer knowledge about classes and properties. For example, it can describe subclass relationships, equivalent classes and restrictions on relationships.

**Example** Suppose the ontology states:

1. Every **Doctor** is a **Person**.
2. Alice is a **Doctor**.

An OWL reasoner can infer:

$$\boxed{\text{Alice is a Person.}}$$

Using class-membership notation:

$$\begin{aligned} &\text{Doctor} \subseteq \text{Person}, \\ &\text{Alice} \in \text{Doctor} \end{aligned} \quad \Longrightarrow \quad \text{Alice} \in \text{Person}.$$

The conclusion need not be stated separately: it follows from the subclass relationship and Alice's class membership.

**Remember** RDF represents facts as triples. OWL adds formal descriptions of classes and properties that allow a reasoner to derive additional knowledge.

## 4.9 Knowledge Representation Using Scripts

**Definition 4:** Script is a structured representation describing a stereotyped sequence of events in a familiar situation (e.g. "dining at a restaurant"), used to help systems fill in unstated details.

A script has: **entry conditions**, **roles** (customer, waiter), **props** (menu, food), and an ordered **scene** sequence (enter → order → eat → pay → exit).

## 4.10 Common Pitfalls

- Confusing $P \to Q$ with $Q \to P$ (the converse) – they are **not** logically equivalent.
- Forgetting that $\forall$ pairs with $\to$ and $\exists$ pairs with $\wedge$ while translating English to FOL.
- Applying resolution directly without first converting sentences to **CNF**.
- In unification, forgetting the **occurs-check** (a variable can't unify with a term containing itself).
- Mixing up forward chaining (data-driven, good for many possible goals) with backward chaining (goal-driven, good for a single specific query).

## 4.11 Practice Problems for Self-Study

**Solved Problem 13:** Practice – validity check using truth table. Check whether the argument is valid: Premises $P \to Q$, $\neg Q$; Conclusion $\neg P$.

*Solution:*

| $P$ | $Q$ | $P \to Q$ | $\neg Q$ | $\neg P$ |
|---|---|---|---|---|
| T | T | T | F | F |
| T | F | F | T | F |
| F | T | T | F | T |
| F | F | T | T | T |

The argument is valid **only if**, in every row where *both* premises ($P \to Q$ and $\neg Q$) are True, the conclusion ($\neg P$) is also True. That happens only in row 4 (the only row where both premises hold), and there $\neg P$ is True. **Hence the argument (Modus Tollens) is valid.**

**Solved Problem 14:** Practice – CNF conversion. Convert $P \leftrightarrow (Q \vee R)$ to CNF.

*Hint:* first eliminate $\leftrightarrow$ to get $(P \to (Q \vee R)) \wedge ((Q \vee R) \to P)$, then eliminate each $\to$, then distribute.
*Final answer:* $(\neg P \vee Q \vee R) \wedge (\neg Q \vee P) \wedge (\neg R \vee P)$.

**Solved Problem 15:** Practice – FOL translation and resolution. Premises: "All cats are mammals" and "Tom is a cat". Prove "Tom is a mammal" using resolution.

*Hint:* $Cat(x) \to Mammal(x)$, $Cat(Tom)$, negate goal $\neg Mammal(Tom)$, unify with $\{x/Tom\}$, resolve twice to reach the empty clause (structure identical to the Socrates example above).

## 4.12 Cheat Sheet

| Concept | Key fact |
|---|---|
| KR system | $\mathcal{K} = (\mathcal{S}, \mathcal{L}, \mathcal{I}, \mathcal{R})$: symbols, language, interpretation, reasoning |
| KB $\models \phi$ / KB $\vdash \phi$ | true in all models / derivable by rules. Sound: $\vdash \Rightarrow \models$; complete: $\models \Rightarrow \vdash$ |
| Refutation | KB $\models \phi$ iff KB $\cup \{\neg\phi\}$ unsatisfiable |
| Taxonomy | is-a DAG; LCA = deepest common ancestor |
| Frame / semantic net | slots + inheritance with overriding / labelled graph of concepts |
| Ontology | $\mathcal{O} = (\mathcal{C}, \mathcal{R}, \mathcal{I}, \mathcal{A})$; RDF triple $(S, P, O)$; OWL adds logic |
| Knowledge graph | $\mathcal{G} = (V, E, L)$; adjacency matrix $A_{ij} \in \{0, 1\}$ |
| Bayesian network | $P(X_1..X_n) = \prod_i P(X_i \mid \text{Parents}(X_i))$ |
| Implication | $p \to q \equiv \neg p \vee q \equiv \neg q \to \neg p$ |
| De Morgan | $\neg(p \wedge q) \equiv \neg p \vee \neg q$; $\neg(p \vee q) \equiv \neg p \wedge \neg q$ |
| FOL patterns | "All $A$ are $B$": $\forall x(A \to B)$. "Some $A$ are $B$": $\exists x(A \wedge B)$. $\neg\forall \equiv \exists\neg$, $\neg\exists \equiv \forall\neg$ |
| Forward chaining | data-driven, fire rules from known facts; Horn KB; sound & complete |
| Backward chaining | goal-driven, prove sub-goals recursively; Prolog |
| Resolution | $\frac{p \vee A,\ \neg p \vee B}{A \vee B}$; refutation-complete; derive $\square$ |
| Unification | find MGU $\theta$; fail on clash or occurs check |
| SAT | NP-complete; DPLL, CDCL |
| Bayes | $P(H \mid E) = \frac{P(E\mid H)P(H)}{P(E)}$ |
| Default logic | $\frac{A:B}{C}$ – if $A$ and $B$ consistent, conclude $C$ |
| CBR | $\arg\min_{C_i} d(P, C_i)$; Retrieve–Reuse–Revise–Retain |